The pxPRP package

Property maps with key/value lists for LaTeX2e

Scvria
P3CTeX / UOC PEC Repository

2025/02 – Version 0.2

Abstract

pxPRP provides a simple hashmap-like abstraction on top of expl3 property lists. Each logical *object* is identified by a short name (e.g. PATATA); within an object, keys hold either a scalar value or a comma-separated list. The package keeps an internal registry so that users refer to objects by id instead of internal expl3 variable names. It is designed for use from plain LaTeX2e and from other packages (e.g. pxUML).

Contents

1 Introduction **1**

2 Loading the package **2**

3 Concepts **2**

4 User command reference **2**

4.1 Object and key management 2
4.2 Scalar values 3
4.3 List operations 3
4.4 Indexed list access 4
4.5 Conditionals 4

5 Minimal example **4**

6 Change history **5**

1 Introduction

pxPRP manages *logical objects*: each object has an identifier (a short name) and a set of *keys*. For each key you can store either:

- a **scalar** value (a single token list), or
- a **comma-separated list** of items (treated as a list for append/set/length operations).

All data are stored in `expl3 prop` variables; the package maintains a global registry mapping object ids to those variables so that you never need to use internal names in your document or in higher-level packages.

2 Loading the package

```
\usepackage{pxPRP}
```

The package requires `expl3`, `xparse`, `l3keys2e`, and `color` (the last is required by the package dependencies).

3 Concepts

Object A logical container identified by a string id (e.g. `MyObj`). Created with `\pxNewObject` or implicitly when you first set a key for that id.

Key A string key within an object. Keys are created when you first assign a value or append to a list.

Scalar value A single value stored under a key (e.g. a flag `true/false` or a short text). Use `\pxSet` and `\pxGetScalar`.

List value A key can instead hold a comma-separated list of items. Use `\pxPutValue`, `\pxListSet`, `\pxListAll`, `\pxGet OBJ.key[i]`, etc.

4 User command reference

4.1 Object and key management

`\pxNewObject` Creates (or ensures existence of) a logical object with the given id. If the object already exists, nothing is cleared.

```
\pxNewObject{<object-id>}
```

Example: `\pxNewObject{PATATA}`.

`\pxClearObject` Removes the object and all its keys from the global registry. The object id can be reused afterwards.

```
\pxClearObject{<object-id>}
```

`\pxClearKey` Removes a single key (and its value) from an object. The key can be set again later.

```
\pxClearKey{<object-id>}{<key>}
```

4.2 Scalar values

`\pxSetValue` Sets the scalar value for a key. Any previous value (scalar or list) is replaced. The value may contain commas.

```
\pxSetValue{<object-id>}{<key>}{<value>}
```

`\pxGetScalar` Expands to the scalar value stored for the key. If the key or object does not exist, the result is empty. Use this when the key holds a single value (e.g. a flag or label), not a list.

```
\pxGetScalar{<object-id>}{<key>}
```

4.3 List operations

`\pxPutValue` Appends one item to the comma-separated list stored at the key. If the key did not exist, a new list containing only that item is created.

```
\pxPutValue{<object-id>}{<key>}{<item>}
```

The third argument can contain commas (it is one "item" as a whole).

`\pxListSet` Replaces the entire list at the key with the given comma-separated list.

```
\pxListSet{<object-id>}{<key>}{<list>}
```

`\pxListPutRight` Appends a comma-separated list to the right of the existing list at the key.

```
\pxListPutRight{<object-id>}{<key>}{<list>}
```

`\pxListPutLeft` Prepends a comma-separated list to the left of the existing list at the key.

```
\pxListPutLeft{<object-id>}{<key>}{<list>}
```

`\pxListClear` Clears the list at the key but keeps the key present (empty list). Contrast with `\pxClearKey`, which removes the key entirely.

```
\pxListClear{<object-id>}{<key>}
```

`\pxListAll` Typesets all items of the list at the key, separated by the given separator. Default separator is `, `.

```
\pxListAll{<object-id>}{<key>}[<separator>]
```

Example: `\pxListAll{OBJ}{nums}[ - ]`.

`\pxListLen` Expands to the number of items in the list at the key (0 if the key is missing or empty).

```
\pxListLen{<object-id>}{<key>}
```

4.4 Indexed list access

\pxGET Compact syntax to get the <index>-th item (1-based) of the list at <key> in object <object-id>. Space after \pxGet is required; the syntax is OBJ.key[index].

```
\pxGET <object-id>.<key>[<index>]
```

Example: \pxGet PATATA.k1[2].

\pxPRINT Same as \pxGet but typesets a bold label object.key [index] : before the value.

```
\pxPRINT <object-id>.<key>[<index>]
```

4.5 Conditionals

\pxIfTrueTF Evaluates the scalar value at the key as a boolean: empty or the literal false is false; anything else is true. Executes the true or false branch accordingly.

```
\pxIfTrueTF{<object-id>}{<key>}{<true code>}{<false code>}
```

\pxIfObjectExistsTF Executes the first branch if the object id is registered, otherwise the second.

```
\pxIfObjectExistsTF{<object-id>}{<true code>}{<false code>}
```

\pxIfKeyExistsTF Executes the first branch if the key exists in the object, otherwise the second.

```
\pxIfKeyExistsTF{<object-id>}{<key>}{<true code>}{<false code>}
```

5 Minimal example

```
\usepackage{pxPRP}
\begin{document}
\pxNewObject{OBJ}
\pxSetValue{OBJ}{name}{MyObject}
\pxPutValue{OBJ}{items}{a}
\pxPutValue{OBJ}{items}{b}
Name: \pxGetScalar{OBJ}{name}.
Items: \pxListAll{OBJ}{items}.
First item: \pxGET OBJ.items[1].
Length: \pxListLen{OBJ}{items}.
\end{document}
```

6 Change history

v0.2 Added `\pxGetScalar`, `\pxListClear`; fixed scalar getter for missing keys; documentation.

v0.1 Initial release: objects, keys, scalar/list storage, conditionals, `\pxGet`/`\pxPrint` syntax.